

박수민_0612_FND_제어하기 과제

```
# main.c
...

/*
 * 02.FND_CONTROL_MY.c
 *
 * Created: 2026-06-12 오후 1:20:39
 * Author : kccistc
 */
#define F_CPU 16000000UL // 16MHz
#include <avr/io.h> // PORTA PORTD 등의 I/O register 들
이 들어있음.
#include <util/delay.h>

#include "button.h"

extern void init_button(void);
extern int get_button(int button_num, int button_pin);

extern int fnd_main();
extern int init_fnd();

#define MODE_CLOCK 0 /* 분:초 시계 */
#define MODE_CHRONO 1 /* 초시계 */
#define MODE_SW 2 /* 스탑워치 */

int main(void)
{
    init_fnd();
    init_button();

    while (1)
    {
        fnd_main();
    }
}
...
```

```

#button.c
...

/*
 * button.c
 *
 * Created: 2026-06-12 오후 1:22:26
 * Author: kccistc
 */

#include "button.h"          // "xx.h" -> 같은 directory에 있는 파일을 가져오겠다

void init_button(void);
int get_button(int button_num, int button_pin);

// 버튼 초기화 방향설정
void init_button(void)
{
    BUTTON_DDR &= ~(1 << BUTTON0PIN | 1 << BUTTON1PIN | 1 << BUTTON2PIN |
1 << BUTTON3PIN);
}

// ex) BUTTON0PIN 3
// button 을 눌렀다 떼면 : 1 리턴
// button 이 idle 상태   : 0 리턴

int get_button(int button_num, int button_pin)
{
    // static 지역변수에 static 을 선언하면 함수를 빠져나와 다시 들어가도 이전값을 유지
한다.
    static unsigned char button_status[BUTTON_NUMBER] =
    {
        BUTTON_RELEASE,    BUTTON_RELEASE,    BUTTON_RELEASE    ,
BUTTON_RELEASE
    };

    int current_state;

    // 1. 버튼을 읽는다.
    current_state = BUTTON_PIN & (1 << button_pin);

    //2. 버튼 상태 check
    if (current_state && button_status[button_num] == BUTTON_RELEASE) // 버튼이
처음 눌러진 상태

```

```

{
    _delay_ms(20); // noise 가 지나가기를 기다림
    button_status[button_num] = BUTTON_PRESS;
    return 0;      // 아직은 완전히 눌렀다 떼 상태가 아님
}

else if (button_status[button_num] == BUTTON_PRESS && current_state ==
BUTTON_RELEASE)
{
    // 버튼이 이전에 눌러진 상태 + 지금은 떼어진 상태
    // 다음 버튼을 체크하기 위해 초기화
    button_status[button_num] = BUTTON_RELEASE;
    _delay_ms(20);
    return 1;      // 완전히 1번 눌렀다 떼 상태로 인정한다.
};

return 0;         // 버튼이 open 상태
}
...

```

```

#button.h
...

/*
 * button.h
 *
 * Created: 2026-06-12 오후 1:23:02
 * Author: kccistc
 */

// #ifndef BUTTON_H_          // BUTTON_H_라는 이름이 정의가 되어있지 않다면, 정의를
해라
// #define BUTTON_H_          // 그래서 정의한 것
// #endif /* BUTTON_H_ */

#define F_CPU 16000000UL      // 16MHz
#include <avr/io.h>            // PORTA, PORTB 등 I/O 관련 reg
#include <util/delay.h>

// 아래는 pull down 저항(버튼) 기준으로 작성

#define BUTTON_DDR DDRD
#define BUTTON_PIN PIND      // PORTD를 읽는 register 5V:1 0V:0

#define BUTTON0PIN 3         // PORTD.3
#define BUTTON1PIN 4         // PORTD.4
#define BUTTON2PIN 5         // PORTD.5
#define BUTTON3PIN 6         // PORTD.6

#define BUTTON0 0            // PORTD.3의 가상 index (software number)
#define BUTTON1 1            // PORTD.4의 가상 index (software number)
#define BUTTON2 2            // PORTD.5의 가상 index (software number)
#define BUTTON3 3            // PORTD.6의 가상 index (software number)

#define BUTTON_NUMBER 4      // 버튼의 갯수

#define BUTTON_PRESS 1        // 버튼을 누르면 high(active-high)
#define BUTTON_RELEASE 0      // 버튼을 땀 상태(low)

...

```

```

#fnd.c
...

/*
 * fnd.c
 *
 * Created: 2026-06-12 오후 1:22:04
 * Author: kccistc
 */

#include "fnd.h"
#include "button.h"

/* —— 모드 정의 —— */
#define MODE_CLOCK    0    // MM.SS 시계 (도트 1초마다 깜빡임)
#define MODE_COUNT    1    // 초시계      (오른쪽 초 카운트 + 왼쪽 원형 애니메이션)
#define MODE_SW       2    // 스톱워치   (버튼1로 start/stop)

#define ANIM_PERIOD   100 // 원형 애니메이션 한 칸 이동 주기(ms)

uint32_t ms_count      = 0;    // 시계용 ms
uint32_t sec_count     = 0;    // 시계용 sec
uint8_t  dot_display   = 0;    // 시계 가운데 도트
uint32_t cnt_ms        = 0;    // 초시계용 ms
uint32_t sw_ms         = 0;    // 스톱워치 누적 ms
uint8_t  anim_step     = 0;    // 초시계 원형 애니메이션 위치 (0~7)

extern int get_button(int button_num, int button_pin);

int fnd_main();
void init_fnd();
void fnd_display();
void fnd_display_clock(uint32_t total_sec, uint8_t dot_on);
void fnd_display_stopwatch(uint32_t total_ms);
void fnd_display_count(uint32_t total_sec, uint8_t step);
void fnd_all_off();

static const uint8_t fnd_font[] = {
    0xC0, 0xF9, 0xA4, 0xB0, 0x99,    /* 0  1  2  3  4  */
    0x92, 0x82, 0xD8, 0x80, 0x98,    /* 5  6  7  8  9  */
    0x40, 0x79, 0x24, 0x30, 0x19,    /* 0. 1. 2. 3. 4.  */
    0x12, 0x02, 0x58, 0x00, 0x18     /* 5. 6. 7. 8. 9.  */
};

```

```

int fnd_main(void)
{
    uint8_t mode = MODE_CLOCK;
    uint8_t sw_run = 0;           // 스톱워치 동작 여부

    ms_count = 0; sec_count = 0; dot_display = 0; sw_ms = 0; anim_step = 0;

    while (1)
    {
        switch (mode)
        {
            case MODE_CLOCK:
                fnd_display_clock(sec_count, dot_display); break;
            case MODE_COUNT:
                fnd_display_count(cnt_ms / 1000, anim_step); break;
            case MODE_SW:
                fnd_display_stopwatch(sw_ms); break;
        }

        _delay_ms(1);

        // 시간 진행
        ms_count++;
        if (ms_count >= 1000)
        {
            ms_count = 0;
            sec_count++;
        }

        if (ms_count == 500)           // 딱 500ms 시점에만 한 번 토글
        {
            dot_display = !dot_display;
        }

        if (mode == MODE_COUNT)
        {
            cnt_ms++;
            if (cnt_ms % ANIM_PERIOD == 0)
                anim_step = (anim_step + 1) % 8;

            // 초시계 자동 증가
            // 일정 주기마다
            // 원형 애니
        }

        // 메이션 한 칸 이동

        if (mode == MODE_SW && sw_run)    sw_ms++;           // 스톱워치는 달리
    }
}

```

는 중에만

```
/* 3) 버튼0 : 모드 전환 */
if (get_button(BUTTON0, BUTTON0PIN))
{
    switch (mode)
    {
        case MODE_CLOCK:
            mode = MODE_COUNT; cnt_ms = 0;
            break; // 초시계 시작(0부터 자동)

        case MODE_COUNT:
            mode = MODE_SW;
            sw_ms = 0; sw_run = 0;
            break; // 스톱워치(정지 상태로 진입)

        case MODE_SW:    mode = MODE_CLOCK;
            break; // 시계로
    }
}

// 버튼1 : 스톱워치 모드에서만 토글
if (get_button(BUTTON1, BUTTON1PIN))
{
    if (mode == MODE_SW)
        sw_run = !sw_run;
}

// 버튼2 : 전체 리셋 → 모든 값 0, 스톱워치 정지, 시계 화면으로
if (get_button(BUTTON2, BUTTON2PIN))
{
    if (mode == MODE_SW)
    {
        if (sw_run == 1)           // 달리는 중 → 리셋+정지
        {
            sw_ms = 0;
            sw_run = 0;
        }
        else if (sw_ms == 0)       // 리셋된 상태 → 시작
        {
            sw_run = 1;
        }
    }
}
```

```

        else // 정지만 된 상태(버튼1로 멈춘
            {
                sw_ms = 0;
                sw_run = 0;
            }
        }
        else // 다른 모드 → 전체 리셋
        {
            ms_count = 0; sec_count = 0; dot_display = 0;
            cnt_ms = 0; anim_step = 0;
            sw_ms = 0; sw_run = 0;
        }
    }

}

//init_fnd();

//while(1)
//{
    //fnd_display();
    //_delay_ms(1);
    //ms_count++;
    //if (ms_count >= 1000) // 1000ms ----> 1sec
    //{
        //ms_count = 0;
        //sec_count++;
        //dot_display = !dot_display;
    //}
//}
return 0;
}

void init_fnd(void)
{
    FND_DATA_DDR = 0xff; // 출력모드로 설정한다.
    FND_DIGIT_DDR |= 1 << FND_DIGIT_D1 | 1 << FND_DIGIT_D2 | 1 <<
    FND_DIGIT_D3 | 1 << FND_DIGIT_D4;

    // FND를 전체 off 하는 작업
    #if 1 // common anode 방식

```



```

        FND_DATA_PORT = 0xff;

#else                // common cathode 방식

        FND_DATA_PORT ~= 0xff;

#endif

        FND_DIGIT_PORT      &=
~((1<<FND_DIGIT_D1)|(1<<FND_DIGIT_D2)|(1<<FND_DIGIT_D3)|(1<<FND_DIGIT_D4));

    }

void fnd_all_off(void)
{
    FND_DIGIT_PORT &= ~((1 << FND_DIGIT_D1) | (1 << FND_DIGIT_D2) | (1 <<
FND_DIGIT_D3) | (1 << FND_DIGIT_D4));
    FND_DATA_PORT   = 0xFF;
}

void fnd_display(void)
{
    static int digit_select = 0;                // 자릿수 선택
    택

    switch(digit_select)
    {
        case(0):
            // 1단위
            #if 1
                FND_DIGIT_PORT = 0x80;          / /
anode
            #else
                FND_DIGIT_PORT = ~0x80;
            // cathode
            #endif
            FND_DATA_PORT = fnd_font[sec_count%10]; // 0~9
            break;

            case(1):

```

```

// 10단위
    #if 1
        FND_DIGIT_PORT = 0x40;
    #else
        FND_DIGIT_PORT = ~0x40;
    #endif
    FND_DATA_PORT = fnd_font[sec_count/10%6]; // 0~59
    break;

    case(2):
// 100단위
    #if 1
        FND_DIGIT_PORT = 0x20;
    #else
        FND_DIGIT_PORT = ~0x20;
    #endif
    FND_DATA_PORT = fnd_font[sec_count/60%10];
    break;

    case(3):
// 1000단위
    #if 1
        FND_DIGIT_PORT = 0x10;
    #else
        FND_DIGIT_PORT = ~0x10;
    #endif
    FND_DATA_PORT = fnd_font[sec_count/600%6]; // 0~59
    break;
}
digit_select = (digit_select + 1) % 4; // 다음 표시할 자리수
}

void fnd_display_count(uint32_t total_sec, uint8_t step)
{
    static uint8_t d = 0;

    uint8_t sec1 = total_sec % 10;
    uint8_t sec10 = (total_sec / 10) % 6;

    /* 각 단계가 어느 칸(자리수), 어느 세그먼트인지 */
    static const uint8_t anim_digit[8] = { 0x20, 0x10, 0x10, 0x10, 0x10, 0x20, 0x20,
0x20 };
    static const uint8_t anim_seg[8] = { 0xFE, 0xFE, 0xDF, 0xEF, 0xF7, 0xF7,

```

0xFB, 0xFD };

```
switch (d)
{
    case 0: // 초1
        FND_DIGIT_PORT = 0x80; FND_DATA_PORT = fnd_font[sec1]; break;
    case 1: // 초10
        FND_DIGIT_PORT = 0x40; FND_DATA_PORT = fnd_font[sec10]; break;
    case 2: // circle anim
        FND_DIGIT_PORT = 0x20;
        FND_DATA_PORT = (anim_digit[step] == 0x20) ? anim_seg[step] : 0xFF;
        break;
    case 3: // circle anim
        FND_DIGIT_PORT = 0x10;
        FND_DATA_PORT = (anim_digit[step] == 0x10) ? anim_seg[step] : 0xFF;
        break;
}
d = (d + 1) % 4;
}
```

```
void fnd_display_clock(uint32_t total_sec, uint8_t dot_on)
{
```

```
    static uint8_t d = 0;
```

```
    uint8_t sec1 = total_sec % 10;
```

```
    uint8_t sec10 = (total_sec / 10) % 6;
```

```
    uint8_t min1 = (total_sec / 60) % 10;
```

```
    uint8_t min10 = (total_sec / 600) % 6;
```

```
switch (d)
```

```
{
```

```
    case 0: // 초 1의 자리 (오른쪽 끝)
```

```
        FND_DIGIT_PORT = 0x80;
```

```
        FND_DATA_PORT = fnd_font[sec1];
```

```
        break;
```

```
    case 1: // 초 10의 자리
```

```
        FND_DIGIT_PORT = 0x40;
```

```
        FND_DATA_PORT = fnd_font[sec10];
```

```
        break;
```

```
    case 2: // 분 1의 자리 (도트 위치)
```

```
        FND_DIGIT_PORT = 0x20;
```

```
        FND_DATA_PORT = fnd_font[dot_on ? min1 + 10 : min1];
```

```

        break;
    case 3:                                     // 분 10의 자리 (왼쪽 끝)
        FND_DIGIT_PORT = 0x10;
        FND_DATA_PORT  = fnd_font[min10];
        break;
    }
    d = (d + 1) % 4;
}

/* SS.CC 표시 (CC = 1/100초 = 10ms 단위). 자릿수 순서 동일: */
void fnd_display_stopwatch(uint32_t total_ms)
{
    static uint8_t d = 0;

    uint32_t cs    = total_ms / 10;           // 10ms(센티초) 단위로 변환
    uint8_t  cs1    = cs % 10;               // 10ms 자리
    uint8_t  cs10   = (cs / 10) % 10;        // 100ms 자리
    uint8_t  sec1   = (cs / 100) % 10;       // 초 1의 자리
    uint8_t  sec10  = (cs / 1000) % 10;      // 초 10의 자리

    switch (d)
    {
        case 0:                                 // 10ms 자리 (오른쪽 끝)
            FND_DIGIT_PORT = 0x80;
            FND_DATA_PORT  = fnd_font[cs1];
            break;

        case 1:                                 // 100ms
            FND_DIGIT_PORT = 0x40;
            FND_DATA_PORT  = fnd_font[cs10];
            break;

        case 2:                                 // 1초대
            FND_DIGIT_PORT = 0x20;
            FND_DATA_PORT  = fnd_font[sec1 + 10];
            break;

        case 3:                                 // 10초대
            FND_DIGIT_PORT = 0x10;
            FND_DATA_PORT  = fnd_font[sec10];
            break;
    }
    d = (d + 1) % 4;
}

```

```
}  
...
```

```
#fnd.h  
...
```

```
/*  
 * fnd.h  
 *  
 * Created: 2026-06-12 오후 1:22:14  
 * Author: kccistc  
 */
```

```
#define F_CPU 16000000UL  
#include <avr/io.h>  
#include <util/delay.h>
```

```
#define FND_DATA_DDR DDRC  
#define FND_DATA_PORT PORTC
```

```
#define FND_DIGIT_DDR DDRB // Data Direction Register B 포트 B 핀의 입출력 방향 설정  
#define FND_DIGIT_PORT PORTB // 자릿수  
#define FND_DIGIT_D1 4  
#define FND_DIGIT_D2 5  
#define FND_DIGIT_D3 6  
#define FND_DIGIT_D4 7  
...
```